

실습문제: 6장. 학습 관련 기술들(2)

신경망

학번: 202404252 이름: 원종우

1. [가중치 초기값] 가중치의 초기값을 주는 기본적인 아이디어는, 가중치마다 서로 다른 작은 숫자들을 할당하는 데, 얼마나, 어떻게 작게 줄 것인가가 문제이다. 교재와 common/multi_layer_net.py 코드를 참고하여 다음에 적절히 답하시오.

(1) 코드에서 신경망 모델의 계층들(Layer)을 구성한 부분을 찾아 옮겨 붙이시오.

```
activation_layer = {'sigmoid': Sigmoid, 'relu': Relu}

self.layers = OrderedDict()
for idx in range(1, self.hidden_layer_num + 1):
    self.layers['Affine' + str(idx)] = Affine(self.params['W' + str(idx)],
    self.params['b' + str(idx)])
    self.layers['Activation_function' + str(idx)] =
activation_layer[activation]()

idx = self.hidden_layer_num + 1
self.layers['Affine' + str(idx)] = Affine(self.params['W' + str(idx)],
self.params['b' + str(idx)])
```

(2) 다음 빈 칸에 적절히 답하시오.

가중치 초기값 설정 방법	파이썬 코드(py 파일 참조)	권장 사용법(교재 참조)
std=0.01	<pre>self.params['W' + str(idx)] = scale * np.random.randn(all_size_list[idx] - 1, all_size_list[idx])</pre>	전통적인 초기화 방법
Xavier	<pre>scale = np.sqrt(2.0 / all_size_list[idx] - 1))</pre>	Sigmoid, Tanh와 잘 맞음
He	<pre>scale = np.sqrt(1.0 / all_size_list[idx] - 1))</pre>	Relu 계열과 잘 맞는다

(3) 코드에서 편향 b는 어떻게 초기화 했는가?

```
self.params['b' + str(idx)] = np.zeros(all_size_list[idx])
```

- 각 층의 편향 벡터는 층의 노드 수만큼 0으로 초기화, np.zeros()를 사용하여 완전한 영 벡터로 설정하였다.

(4) [그림 6-14]를 보고 세 방법 중, He 초기값을 사용하는 것이 가장 좋다고 말할 수 있는 이유는?

- 깊은 신경망에서도 학습 속도와 정확도가 더 잘 나오기 때문이다. 실험적으로 ReLU 기반 신경망에서는 Xavier보다 빠르게 수렴하고 정확도도 더 잘 나오는 경우가 많다.

(5) 교재에서 제안하는 초기화 방법의 핵심을 간단히 적으시오.

- 활성화 함수의 특성에 맞춰 가중치의 분산을 조절해 초기화해야 학습이 안정된다.

2. weight_init_compare.py 파일에 대해 물음에 답하시오.

(1) 신경망 구조를 설명하시오.

- 입력층 1개, 은닉층 4개, 출력층 1개로 구성된 다층 퍼셉트론 구조이다. 입력층은 MNIST 이미지(784 차원)를 받으며, 각 은닉층은 100개의 뉴런으로 이루어진 완전연결층과 활성화 함수가 이어진 형태로 구성된다. 마지막 출력층은 10개의 뉴런을 가지며 Softmax 함수를 사용하여 10개 클래스에 대한 확률을 출력한다. 전체 구조는 $784 \rightarrow 100 \rightarrow 100 \rightarrow 100 \rightarrow 100 \rightarrow 10$ 으로 구성된다.

(2) optimizer로 Adam을 사용하도록 코드를 수정하고, Xavier, He 초기화시 손실함수 값이 0에 수렴하도록 학습률을 적절히 조정하시오. 코드와 그래프 결과를 옮겨 붙이시오.

```
import sys, os
import numpy as np
import matplotlib.pyplot as plt

sys.path.append(os.path.join(os.path.dirname(__file__), '..'))

from data.mnist import load_mnist
from common.util import smooth_curve
from common.multi_layer_net import MultiLayerNet
from common.optimizer import Adam # Adam 사용

# 0. MNIST 데이터 =====
(x_train, t_train), (x_test, t_test) = load_mnist(normalize=True)

train_size = x_train.shape[0]
batch_size = 128
max_iterations = 2000

# 1. 실험 설정 =====
weight_init_types = {'Xavier': 'sigmoid', 'He': 'relu'}

optimizer = Adam(lr=0.001) # Xavier, He 둘 다 잘 수렴하는 최적 learning rate

networks = {}
train_loss = {}

for key, weight_type in weight_init_types.items():
    networks[key] = MultiLayerNet(
        input_size=784,
```

```

hidden_size_list=[100, 100, 100, 100],
output_size=10,
weight_init_std=weight_type
)
train_loss[key] = []

```

2. 훈련 시작 =====

```

for i in range(max_iterations):
    batch_mask = np.random.choice(train_size, batch_size)
    x_batch = x_train[batch_mask]
    t_batch = t_train[batch_mask]

    for key in weight_init_types.keys():
        grads = networks[key].gradient(x_batch, t_batch)
        optimizer.update(networks[key].params, grads)

    loss = networks[key].loss(x_batch, t_batch)
    train_loss[key].append(loss)

    if i % 100 == 0:
        print("===== iteration:", i, "=====")
        for key in weight_init_types.keys():
            print(key, ":", networks[key].loss(x_batch, t_batch))

```

3. 손실 그래프 =====

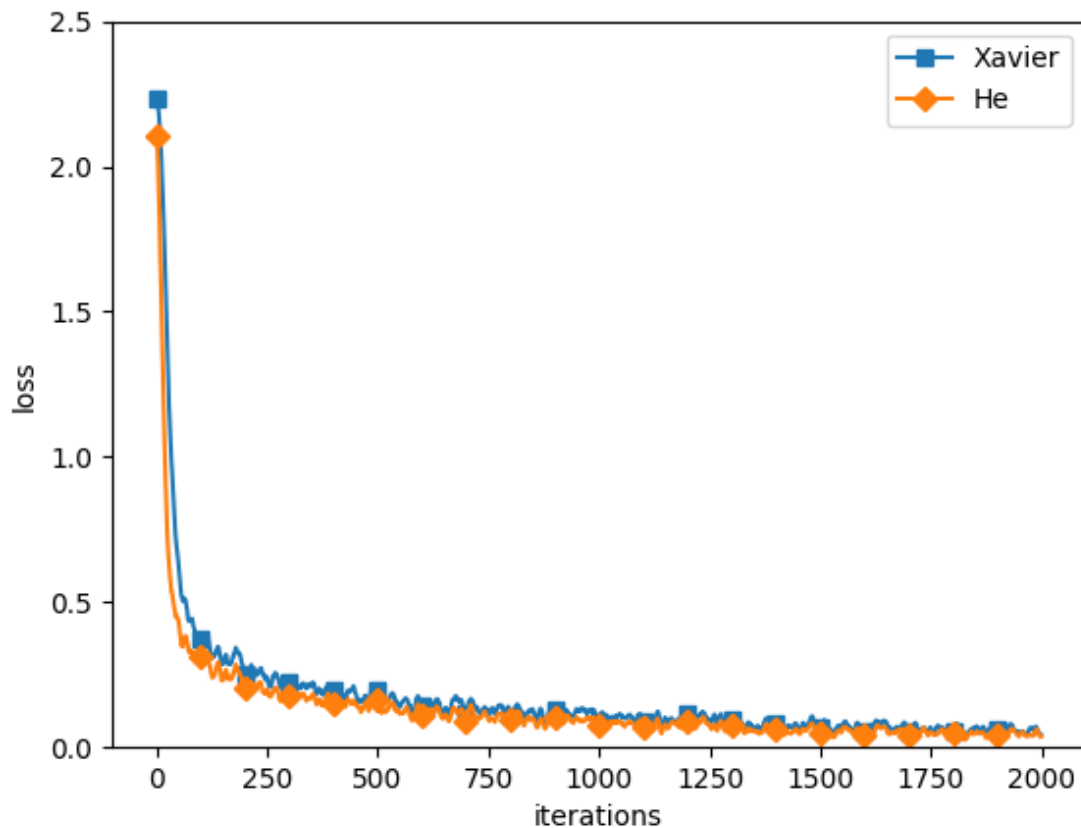
```

markers = {'Xavier': 's', 'He': 'D'}
x = np.arange(max_iterations)

for key in weight_init_types.keys():
    plt.plot(x, smooth_curve(train_loss[key]),
             marker=markers[key], markevery=100, label=key)

plt.xlabel("iterations")
plt.ylabel("loss")
plt.ylim(0, 2.5)
plt.legend()
plt.show()

```



(3) 출력한 그래프는 어떤 점들을 연결하여 그린 그래프인가? 더 정확하게 그리려면 어떻게 하면 되는가? 그럴 필요가 있는가?

- 그래프는 각 iteration에서 계산된 미니배치 손실값을 순서대로 연결한 곡선이며, smooth_curve로 평활화된 값들이다. 더 정확하게 그리려면 전체 데이터(또는 검증 데이터)의 손실을 계산하거나 smoothing을 제거해야 한다. 그러나 초기화 방식의 비교라는 실험 목적에는 미니배치 손실 곡선면으로도 충분하며, 굳이 더 정확하게 그릴 필요는 없다.

3. [배치 정규화] 배치 정규화를 사용하면, [그림 6-19]에서 보듯이, 학습이 빨리 되어 학습시간(파라미터가 수렴하는데 필요한 에폭 수)을 줄일 수 있다(적은 에폭만 학습해도 된다)고 알려져 있다.

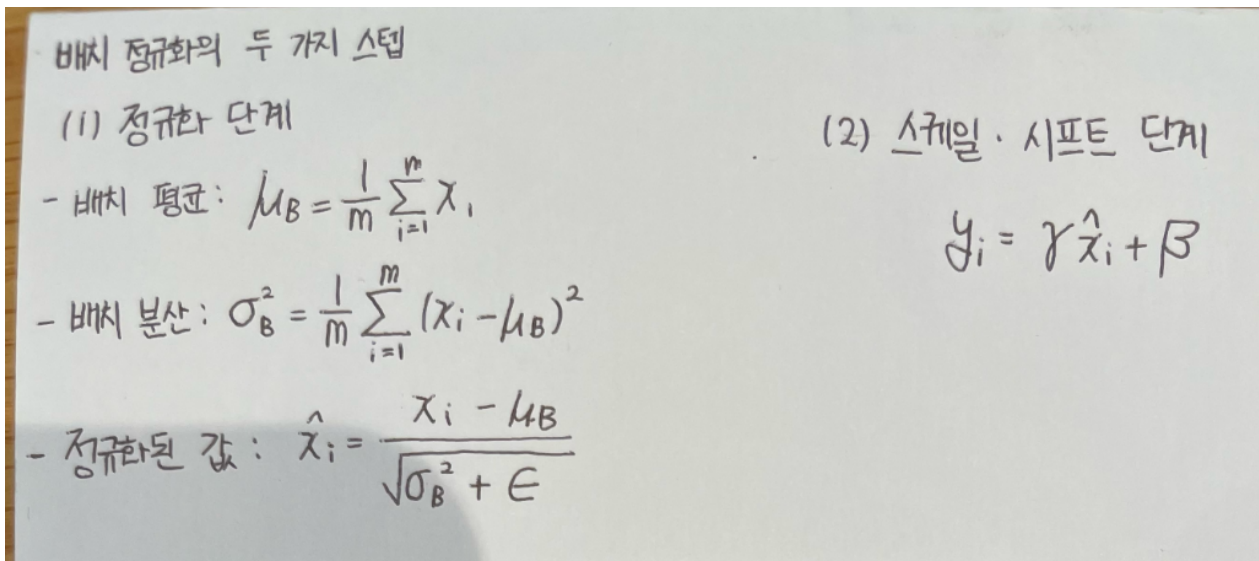
(1) 배치 정규화의 기본 아이디어는 무엇인가?

- 배치 정규화는 각 층의 입력을 미니배치 단위로 평균 0, 분산 1이 되도록 정규화한 후, 학습 가능한 스케일(γ)과 이동(β)을 적용하여 다시 조정하는 방식이다. 이를 통해 층마다의 입력 분포 변화를 줄이고(Internal Covariate Shift 완화), 학습을 안정적으로 빠르게 만들려는 것이 핵심 아이디어이다.

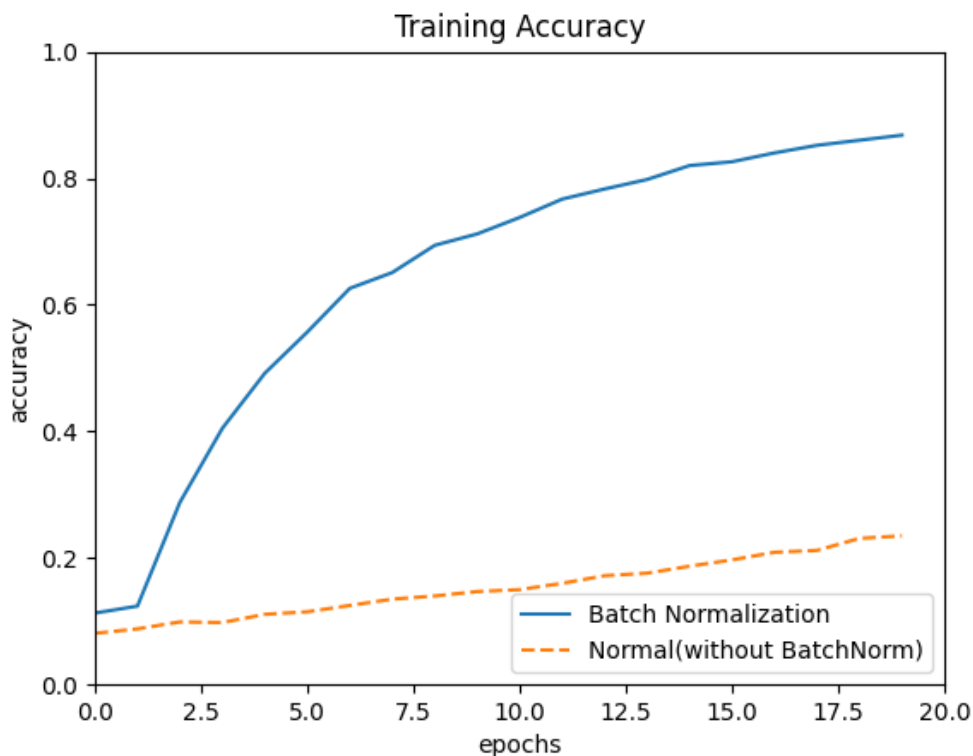
(2) 배치 정규화는 언제(어느 때) 하는가?

- 배치 정규화는 각 층의 선형 변환($Wx + b$) 뒤, 활성화 함수(ReLU 등)를 적용하기 전에 수행한다. 즉, 층에 입력되는 값을 미니배치 단위로 정규화하는 과정에서 사용된다.

(3) 배치 정규화는 두 가지 스텝(step)으로 구성되어 있다. 무엇인가? 필요한 수식을 적으시오



- (4) 배치 정규화에서 데이터를 사용하여 학습하는 매개변수는?
- γ (감마): 정규화된 값을 얼마나 스케일링할지 결정하는 파라미터
 - β (베타): 정규화된 값을 얼마나 **이동(shift)**할지 결정하는 파라미터
- (5) batch_norm_test.py 코드를 실행하고 출력된 그래프를 옮겨 붙이시오.



- (6) 위 그래프를 보고 배치 정규화를 사용할 때 학습이 빠르게 진행된다고 말할 수 있는 이유는?
- 그래프를 보면 같은 에폭 수(0~20) 동안 배치 정규화를 적용한 모델의 정확도는 빠르게 상승하여 80% 이상에 도달한 반면, 배치 정규화를 사용하지 않은 모델의 정확도는 20%대에 머물러 있다. 즉, 같은 학습 시간(에폭 수) 안에 정확도가 훨씬 더 빨리 올라가고 파라미터가 더 빨리 수렴하기 때문에, 배치 정규화를 사용하면 학습이 더 빠르게 진행된다고 말할 수 있다.